

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 17 of 22 — Robustness and Advanced Stats: Extended
Checks, Geocoding Bias, Power Analysis

`extended_robustness_checks.py`,
`geocoding_selection_bias_check.py`, `power_analysis.py`

Sakshi ke liye, Sakshi ki apni research par. Is PDF ki teeno files bilkul teen alag angles se poochti hain ki humari research par kitna bharosa kiya ja sakta hai. Part A, `extended_robustness_checks.py`, poochta hai: 'kya humara control-group DiD result kisi ek district ya ek statistical assumption par zyada depend karta hai?' — aur is file ki apni history poore project ki sabse dramatic kahaniyon mein se ek hai, jahan ek result significant se null, phir null se dobara significant, phir aakhir mein pakke tarike se null ban gaya, jaise-jaise data ki asli galtiyan ek-ek karke pakdi gayi. Part B, `geocoding_selection_bias_check.py`, poochta hai: 'kya 251-village core sample khud, apni geocoding process ki wajah se, kisi taraf biased hai?' Part C, `power_analysis.py`, poochta hai ek bilkul different tarah ka sawaal: 'humare null results ka matlab "koi effect nahi hai" hai, ya "itna chhota effect hai ki humara design use pakad hi nahi sakta"?'

Ye teeno sawaal, alag-alag angles se, ek hi larger cheez achieve karte hain — is research ko sirf 'null result mila' kehne se aage badha kar, 'null result mila, aur humein pata hai wo kitna solid hai, humara sample kitna representative hai, aur humara design kitna chhota effect pakad sakta tha' tak le jaate hain. Ye bilkul waisa hi hai jaise ek doctor sirf 'test negative aaya' na bole, balki ye bhi bataye ki test kitna sensitive tha aur kitna chhota infection wo miss kar sakta tha.

Shuru Karne Se Pehle — Teen Alag Tarah Ki 'Kya Hum Sach Mein Bharosa Kar Sakte Hain' Checks

PDF 14, 15, aur 16 mein humne dekha ki ek significant ya null result mil jaana kaafi nahi hai — us result ko kai alag angles se stress-test karna padta hai. Is PDF ki teeno files usi family ka hissa hain, lekin teen bilkul alag sawaalon ke saath. `Extended_robustness_checks.py` poochta hai 'kya result kisi ek district par ya kisi statistical technicality par depend karta hai' — leave-one-district-out aur randomization inference jaise tareekon se. `Geocoding_selection_bias_check.py` poochta hai 'kya humara sample khud hi, shuru se, kisi taraf jhuka hua hai' — ye ek bilkul alag tarah ka sawaal hai, jo statistical test ke bare mein nahi, balki sample banane ke process ke bare mein hai. Aur `power_analysis.py` poochta hai 'agar hume kuch nahi mila, toh kya wo isliye hai kyunki wahan kuch tha hi nahi, ya isliye kyunki humara design kaafi chhote effects pakad nahi sakta tha.'

EXAMPLE — Ek Metal-Detector Ki Analogy — Teenon Sawaal Ek Saath

Socho ek insaan beach par ek metal detector leke sona dhoondh raha hai, aur use kuch nahi milta. Teen alag sawaal poochne chahiye is 'kuch nahi mila' result ko samajhne ke liye. Pehla: 'kya ye result kisi ek chhote patch par zyada depend karta hai, ya poore beach par consistently kuch nahi mila?' (`extended_robustness_checks.py` ka leave-one-out jaisa). Doosra: 'kya beach ka wo hissa jahan usne dhoonda, poore beach ka representative sample hai, ya kisi wajah se sirf easy-to-reach patches hi cover hue?' (`geocoding_selection_bias_check.py`). Teesra: 'uska metal detector kitna sensitive hai — kya wo ek chhota sikka bhi pakad sakta hai, ya sirf badi cheezein?' (`power_analysis.py`). Teenon sawaalon ka jawaab positive aane par hi 'kuch nahi mila' ek genuinely convincing result banta hai, sirf ek honest attempt nahi.

Part A — `extended_robustness_checks.py`

Ye file is poore 22-PDF series ki sabse dramatic history rakhti hai — ek aisi kahani jahan yehi checks, bina khud badle, teen alag baar, teen alag verdicts diye, kyunki har baar unke neeche ka data khud badalta gaya. Pehle poori file line-by-line padhenge — jo aaj current hai — fir uski poori history dekhenge.

Docstring Aur Imports

```
"""
```

```
Extra robustness checks for the summer-window control-group DiD (Section 4.6):  
1. leave-one-district-out, 2. randomization inference (shuffle treatment  
within district), 3. log1p VIIRS sensitivity, 4. summer-window missingness  
mechanism.
```

statsmodels wouldn't install here so did OLS + cluster-robust/HC3 by hand with numpy - checked it reproduces did_model.py's numbers to 5 sig figs first.

Run from repo root: `python src/analysis/extended_robustness_checks.py`
.....

```
import json
```

```
import numpy as np
import pandas as pd
from scipy import stats
```

Purpose: Docstring seedha 4 checks list karta hai jo ye file karti hai, aur ek zaroori engineering-honesty note bhi deta hai: statsmodels is sandbox mein install nahi ho payi (jo PDF 14 mein `wild_cluster_bootstrap.py` mein bhi dekha tha), isliye poora OLS aur cluster-robust/HC3 covariance haath se, numpy se likha gaya — lekin sirf likh dene se kaam nahi chalta, docstring khud kehta hai ki isse pehle `did_model.py` ke already-committed numbers ke against verify kiya gaya, 5+ significant figures tak match karke, taaki naye check chalane se pehle bharosa ho ki manual implementation sahi hai. Syntax: `import json, numpy as np, pandas as pd, from scipy import stats` — is poore series mein bahut baar dekha gaya standard set. Koi naya import nahi is baar.

Manual OLS Aur Cluster-Robust Sandwich — Ek Chhota Recap, PDF 14 Se

```
def _ols_fit(X, y):
    XtX_inv = np.linalg.inv(X.T @ X)
    beta = XtX_inv @ (X.T @ y)
    resid = y - X @ beta
    return beta, resid, XtX_inv

def _cluster_robust_se(X, resid, XtX_inv, clusters):
    n, k = X.shape
    groups = np.unique(clusters)
    G = len(groups)
    meat = np.zeros((k, k))
    for g in groups:
        idx = clusters == g
        score = X[idx].T @ resid[idx]
        meat += np.outer(score, score)
    adj = (G / (G - 1)) * ((n - 1) / (n - k))
    V = adj * XtX_inv @ meat @ XtX_inv
    return np.sqrt(np.diag(V))
```

Purpose: Ye bilkul PDF 14 ke `wild_cluster_bootstrap.py` mein dekhi gayi `manual_ols()` aur `cluster_robust_se()` jaisi hi hain, sirf naam thoda alag (`_ols_fit`,

`_cluster_robust_se`, aage underscore prefix batata hai ki ye 'internal', sirf isi file ke andar use hone wale helper functions hain). `_ols_fit()` matrix-algebra se beta (coefficients) aur residuals nikaalta hai. `_cluster_robust_se()` har cluster (district) ke apne 'score' ($X.T @ \text{resid}$, ek district ke andar residuals aur predictors ka combined signal) ka outer-product jodkar ek 'sandwich meat' banata hai, fir small-sample correction ($G/(G-1)$ aur $(n-1)/(n-k)$) lagakar final variance-covariance matrix banata hai, jiski diagonal ka square-root standard errors deta hai. Syntax: Underscore-prefixed function names (`_ols_fit`) — Python convention, batata hai ki ye function 'private,' sirf isi module ke andar use ke liye hai, bahar se import karke use karne ka irada nahi hai (halaanki Python isko technically rokta nahi, sirf ek convention/signal hai). `X.shape` — numpy array ka shape (rows, columns) ek tuple mein, $n, k = X.shape$ dono ko ek saath unpack kar leta hai. `np.unique(array)` — ek array mein se sabhi distinct values nikaalna, duplicates hataakar. `np.outer(a, b)` — do vectors ka outer product, ek matrix banata hai jismein har entry $(i,j) = a[i]*b[j]$ hoti hai — PDF 14 mein bhi dekha gaya, sandwich estimator ka core building block.

`_build_design()` — District Fixed Effects Wala Design Matrix

```
def _build_design(df, fe=True):
    n = len(df)
    cols = [np.ones(n), df["treatment"].values.astype(float),
            df["post"].values.astype(float), df["did_term"].values.astype(float)]
    names = ["const", "treatment", "post", "did_term"]
    if fe:
        districts = df["district"].astype("category")
        cats = districts.cat.categories
        for c in cats[1:]: # drop first as reference level
            cols.append((districts == c).astype(float).values)
            names.append(f"district_{c}")
    return np.column_stack(cols), names
```

Purpose: Ye function `did_model.py` (PDF 12) ka wahi DiD design matrix banata hai — constant, treatment, post, did_term (in char columns ke matlab PDF 12 mein poore detail se seekhe the), aur agar `fe=True` (fixed effects), toh district ke liye ek-hot-encoded dummy columns bhi jodta hai, pehla district reference-level ki tarah chhod ke (jaise PDF 12 ke `pd.get_dummies(drop_first=True)` mein bhi dekha tha). Ye function poori file mein har DiD-jaisa test chalaane ke liye reuse hota hai — `run_did()` isko andar se call karta hai. Syntax: `df["col"].astype("category")` — pandas ka categorical data-type, jismein unique values 'categories' ki tarah track hoti hain. `.cat.categories` — un categories ki poori list. `cats[1:]` — pehli category chhod kar baaki sab, Python slicing (index 1 se aakhri tak). `(districts == c).astype(float)` — boolean array (True/False) ko 1.0/0.0 mein badalna, one-hot-encoding ka manual version. `np.column_stack(list)` — multiple 1D arrays ko side-by-side jodkar ek 2D matrix banana, jaise PDF 14 mein bhi dekha gaya tha ek jagah design matrix banaane ke liye.

run_did() — Ek Single, Reusable DiD-Fitting Function

```
def run_did(df, outcome, cluster_col="district"):
    """District-FE DiD with cluster-robust SE, matches did_model.py's spec."""
    valid = df.dropna(subset=[outcome]).copy()
    y = valid[outcome].values.astype(float)
    X, names = _build_design(valid, fe=True)
    beta, resid, XtX_inv = _ols_fit(X, y)
    se = _cluster_robust_se(X, resid, XtX_inv, valid[cluster_col].values)
    idx = names.index("did_term")
    coef, coef_se = beta[idx], se[idx]
    z = coef / coef_se
    p = 2 * stats.norm.sf(np.abs(z))
    return {
        "coef": float(coef), "se": float(coef_se), "p": float(p),
        "ci_lo": float(coef - 1.96 * coef_se), "ci_hi": float(coef + 1.96 * coef_se),
        "n": len(valid), "n_districts": int(valid[cluster_col].nunique()),
    }
```

Purpose: Ye function poori file ki 'workhorse' hai — jo bhi dataframe aur outcome diya jaaye, ye poora DiD model fit karke, sirf did_term (asli causal estimate) ka coefficient, SE, p-value, aur confidence interval ek clean dictionary mein wapas deta hai. Ye reusable design hi is file ko itna compact banata hai — leave-one-out, randomization inference, aur log1p sensitivity, teeno checks isi ek function ko baar-baar, alag-alag dataframes ke saath call karte hain, code duplicate kiye bina. Syntax: names.index("did_term") — ek list mein se ek specific string dhoondh kar uska position (index number) nikaalna. $z = \text{coef} / \text{coef_se}$ — z-statistic, coefficient ko uske apne standard error se divide karke. stats.norm.sf(x) — normal distribution ka 'survival function,' matlab 'is value se zyada extreme value milne ka probability' — Development Log ki apni note (Entry 20) yaad rakhna: ye normal/z-test hai, t-test nahi, kyunki statsmodels ka apna default bhi normal-based hai, aur exact match ke liye yehi zaroori tha.

EXAMPLE — Z-Test Vs T-Test — Ek Chhota Lekin Zaroori Farak

Development Log mein khud likha hua ek subtle point hai jo dhyaan dene layak hai: jab sample size chhota ho (jaise yahan sirf 14 districts), statisticians aksar t-distribution use karte hain normal (z) distribution ke bajaye, kyunki t-distribution ka apna 'fat tails' hota hai jo chhote-sample uncertainty ko better account karta hai. Lekin statsmodels ka apna default cov_type="cluster" wala significance test normal/z-test hi use karta hai, t-test nahi — ye ek design-choice hai, 'sahi' ya 'galat' ka sawaal nahi, lekin agar tum exactly wahi number reproduce karna chahte ho jo statsmodels deta, toh tumhe bhi normal-based hi rehna padega, apne aap se 'zyada correct' lagne wala t-test nahi daal sakte. Yehi wajah hai ki upar wala code stats.norm.sf() use karta hai, stats.t.sf() nahi — chahe intuitively t-test yahan zyada appropriate lage.

1. leave_one_district_out() — Kya Result Kisi Ek District Par Depend Karta Hai?

```
def leave_one_district_out():
    print("=" * 70)
    print("1. LEAVE-ONE-DISTRICT-OUT (summer-window DiD, both outcomes)")
    print("=" * 70)
    df = pd.read_csv(PANEL_PATHS["summer"])
    districts = sorted(df["district"].unique())
    all_rows = []
    summary = {}
    for outcome in ["ndbi", "lights"]:
        rows = []
        for d in districts:
            r = run_did(df[df["district"] != d], outcome)
            r["outcome"] = outcome
            r["dropped_district"] = d
            r["significant"] = r["p"] < 0.05
            rows.append(r)
            print(f" [{outcome}] drop {d:16s} coef={r['coef']:+.5f} p={r['p']:.5f} "
                  f"'{'YES' if r['significant'] else 'no'}")
        all_rows.extend(rows)
        coefs = [r["coef"] for r in rows]
        n_sig = sum(r["significant"] for r in rows)
        summary[outcome] = {
            "coef_min": min(coefs), "coef_max": max(coefs),
            "n_significant_of_14": n_sig,
        }
    print(f" -> {outcome}: coef range [{min(coefs):+.5f}, {max(coefs):+.5f}], "
          f"significant (p<0.05) in {n_sig}/14 district-dropped reruns\n")
    pd.DataFrame(all_rows).to_csv(OUT_LOO_CSV, index=False)
    print(f"Saved {OUT_LOO_CSV}\n")
    return summary
```

Purpose: Is function ka core idea seedha hai: agar summer-window ka DiD result sirf ek particular district ki wajah se significant (ya null) dikh raha hai, toh us district ko hatakar model dobara fit karne se result badal jaayega. Isliye ye function 14 baar (14 districts ke liye) poora model dobara fit karta hai, har baar ek district minus karke, dono outcomes (ndbi, lights) ke liye. Agar result sabhi 14 reruns mein consistent rahe (sabhi significant ya sabhi non-significant), toh koi ek district poore result ko akela drive nahi kar raha — result 'robust' hai is meaning mein. Har run ka coefficient, p-value, aur significance-flag ek CSV mein save hota hai, aur ek chhota summary (coefficient ki range, kitne 14 mein se significant nikle) return hota hai. Syntax: `df[df["district"].unique()` — column ki sabhi distinct values, `sorted()` se alphabetically order mein. `df[df["district"] != d]` — boolean filtering, 'district' column jahan d ke barabar NAHI hai wahi rows rakho — matlab d ko hata do. `all_rows.extend(rows)` — ek list mein doosri poori list ke sabhi elements jodna (`append()` ek single item jodta,

extend() पूरी list). sum(r["significant"] for r in rows) — generator expression के andar boolean values का sum, True को 1 गिन कर total count देता है। f"{d:16s}" — string को 16 characters की fixed width में print करना, table-जैसा alignment के लिये.

2. randomization_inference() — Cluster-Robust Asymptotics पर Bharosa Kiye Bina

```
def randomization_inference():
    print("=" * 70)
    print(f"2. RANDOMIZATION INFERENCE ({N_PERMUTATIONS} permutations, "
          "treatment shuffled within district)")
    print("=" * 70)
    rng = np.random.RandomState(RNG_SEED)
    results = {}
    for outcome in ["ndbi", "lights"]:
        df = pd.read_csv(PANEL_PATHS["summer"])
        observed = run_did(df, outcome)
        obs_coef = observed["coef"]

        # village_id repeats across treated/control so need treatment in the key too
        df["_key"] = df["treatment"].astype(str) + "_" + df["village_id"].astype(str)
        villages = df.drop_duplicates(subset=["_key"])[["_key", "district", "treatment"]]
```

Purpose: Ye check ek bilkul alag philosophy se sochta hai: cluster-robust SE की पूरी theory कुछ asymptotic (bade-sample) assumptions पर तिकी है, जो सिर्फ 14 clusters (districts) के साथ थोड़ी shaky हो सकती है। Randomization inference in assumptions पर bharosa hi nahi karta — ye seedha poochta hai 'agar treatment assignment genuinely random hoti (jo asal mein nahi thi, lekin as-if test के लिये), toh kitni baar itna extreme coefficient सिर्फ chance से milta?' Comment khud zaroori technical detail bataata hai: village_id treated aur control, dono taraf repeat हो सकता है (jaise treated village 5 aur control village 5 dono exist kar sakte hain), isliye ek unique key banane के लिये treatment status aur village_id, dono जोड़ने पड़ते हैं। Syntax: str + "_" + str — string concatenation, दो columns को underscore से जोड़कर एक नया unique-identifier column बनाना. .drop_duplicates(subset=["_key"]) — सिर्फ unique _key values वाली पहली occurrence रakhna, duplicates हटा देना — यहाँ iska use har village का सिर्फ एक row nikaalne के लिये हो रहा है (asli panel mein har village के before/after, dono के rows hain).

```
perm_coefs = []
for _ in range(N_PERMUTATIONS):
    perm = villages.copy()
    for d, grp in perm.groupby("district"):
        perm.loc[grp.index, "treatment"] = rng.permutation(grp["treatment"].values)
    treat_map = dict(zip(perm["_key"], perm["treatment"]))
    df_p = df.copy()
    df_p["treatment"] = df_p["_key"].map(treat_map)
    df_p["did_term"] = df_p["treatment"] * df_p["post"]
```

```

perm_coefs.append(run_did(df_p, outcome)["coef"])

perm_coefs = np.array(perm_coefs)
# two-sided randomization p-value (+1/+1 continuity correction)
p_perm = (np.sum(np.abs(perm_coefs) >= np.abs(obs_coef)) + 1) / (len(perm_coefs) +
1)

```

Purpose: Har ek 2000 permutations mein: villages ke treatment labels ko district ke andar hi randomly shuffle karo (isliye 'within district' — har district ka apna treated/control count same rehta hai, sirf konsa specific village treated hai wo badalta hai — ye important hai kyunki treatment asal mein bhi district-level policy decision thi, poori tarah random nahi). Fir naya treatment_map banao, poore panel mein naye treatment labels daalo, did_term (treatment × post) dobara compute karo, aur poora DiD model dobara fit karo, sirf coefficient store karte hue. 2000 aise coefficients ka collection hi 'null distribution' hai — 'agar treatment random hoti, coefficient kaisi values le sakta tha.' Aakhri mein, in 2000 mein se kitne, apni absolute value mein, asli observed coefficient jitne ya usse zyada extreme hain, wahi randomization p-value hai.

Syntax: perm.groupby("district"): ek loop jo dataframe ko district ke hisaab se groups mein todta hai, har group (d, grp) ek tuple hai — district ka naam aur uska sub-dataframe. perm.loc[grp.index, "treatment"] = ... — specific rows (grp.index se pehchaane gaye) ke 'treatment' column mein naya value daalna. rng.permutation(array) — ek array ki values ko randomly shuffle karna, PDF 13/14/16 mein bhi dekha. dict(zip(a, b)) — do lists ko jodkar dictionary banana, PDF 15 mein bhi is jaisa dekha tha. df["col"].map(dict) — har row ki value ko dictionary mein dhoondhkar corresponding value se replace karna.

3. log_viirs_sensitivity() — Kya Result Kuch Extreme-Bright Villages Ki Wajah Se Hai?

```

def log_viirs_sensitivity():
    print("=" * 70)
    print("3. LOG-TRANSFORMED VIIRS (log1p) SENSITIVITY")
    print("=" * 70)
    treated_only = {}
    for window_label, path in [("full-year", TREATED_PATHS["full_year"]),
                                ("summer-matched", TREATED_PATHS["summer"])]:
        df = pd.read_csv(path)
        core = df[df["is_core_sample"] == True]
        valid = core.dropna(subset=["lights_before", "lights_after"])
        _, raw_p = stats.wilcoxon(valid["lights_after"], valid["lights_before"],
alternative="greater")
        log_before = np.log1p(valid["lights_before"].clip(lower=0))
        log_after = np.log1p(valid["lights_after"].clip(lower=0))
        _, log_p = stats.wilcoxon(log_after, log_before, alternative="greater")
        print(f" Treated-only [{window_label}]: n={len(valid)} raw p={raw_p:.6f} log1p

```

```
p={log_p:.6f}")
    treated_only[window_label] = {"n": len(valid), "raw_p": float(raw_p), "log1p_p":
float(log_p)}
```

Purpose: VIIRS night-lights data ka apna ek statistical quirk hota hai — 'right-skewed' hona, matlab zyadatar villages ki roshni low-medium range mein hoti hai, lekin kuch bahut hi bright outliers (jaise koi town-jaisi jagah) poori distribution ko khinch sakte hain. Ye function check karta hai ki H1 ka lights-change result kahin sirf un handful extreme-bright villages ki wajah se toh nahi mil raha. log1p (log of 1+x) ek standard transformation hai jo aisi skewed values ko 'compress' karta hai — bade numbers ka farak chhota kar deta hai, chhote numbers ka farak zyada-ya-kam waisa hi rakhta hai — taaki extreme values ka poore test par asar kam ho. Syntax: .clip(lower=0) — kisi bhi negative value ko 0 par 'clip' (cap) kar dena, kyunki log(negative) undefined hota hai, aur night-lights readings theoretically kabhi negative nahi honi chahiye (agar sensor-noise se aisa ho bhi jaaye, to 0 treat karna safe hai). np.log1p(x) — log(1+x) ka shortcut function, jo x=0 ke paas numerically zyada stable hai plain log(x) se (jo x=0 par undefined/-infinity hota). List of tuples par loop, teen values (label aur path pehle se pair kiye gaye) unpack karte hue.

```
did = {}
for window in ["full_year", "summer"]:
    df = pd.read_csv(PANEL_PATHS[window])
    df["lights_log1p"] = np.log1p(df["lights"].clip(lower=0))
    r_raw = run_did(df, "lights")
    r_log = run_did(df, "lights_log1p")
    print(f" DiD [{window}]: raw coef={r_raw['coef']:.5f} p={r_raw['p']:.5f} | "
          f"log1p coef={r_log['coef']:.5f} p={r_log['p']:.5f}")
    did[window] = {"raw": r_raw, "log1p": r_log}
print()
return {"treated_only": treated_only, "did": did}
```

Purpose: Bilkul wahi idea, lekin ab control-group DiD ke liye — panel mein ek naya lights_log1p column banao, fir run_did() ko dono versions (raw lights, log1p lights) ke saath call karo, dono windows ke liye. Agar raw aur log1p, dono versions same conclusion dete hain (dono significant ya dono non-significant), toh result outlier-driven nahi hai. Syntax: df["new_col"] = np.log1p(df["col"].clip(lower=0)) — ek naya transformed column banakar dataframe mein add karna, ek hi line mein. Nested dictionary return karna ({"treated_only": ..., "did": ...}) — is poore series mein bahut baar dekha gaya, function ke poore result ko structured, readable format mein dena.

4. summer_missingness() — Ek Check Jo Ab 'Moot' Ho Chuki Hai (Lekin Kyun, Ye Zaroori Hai)

```
def summer_missingness():
    print("=" * 70)
    print("4. SUMMER-WINDOW MISSINGNESS MECHANISM (core sample) -- HISTORICAL")
```

```

print("=" * 70)
summer = pd.read_csv(TREATED_PATHS["summer"])
core = summer[summer["is_core_sample"] == True].copy()
core["summer_valid"] = core["ndbi_before"].notna() & core["ndbi_after"].notna()
n_valid, n_invalid = int(core["summer_valid"].sum()), int(~core["summer_valid"].sum())
print(f" n core = {len(core)}, summer-valid = {n_valid}, summer-invalid = {n_invalid}")

if n_invalid == 0:
    # extraction's complete now (251/251) so nothing to compare against -
    # keeping this check around anyway so the old result stays reproducible
    print(" No summer-invalid villages remain (fresh, complete extraction -- see Entry
22).")
    print(" This check is now moot: nothing left to compare against. Historical result")
    print(" (154 valid / 97 invalid, chi2=71.42 p~3.1e-16, baseline NDBI Mann-Whitney
p=0.00063)")
    print(" is preserved in Development Log Entry 20 and superseded Research Paper
drafts.\n")
    return {
        "n_core": len(core), "n_summer_valid": n_valid, "n_summer_invalid": n_invalid,
        "moot": True, "note": "No missingness remains as of Entry 22's fresh extraction.",
    }

```

Purpose: Ye function bahut interesting hai kyunki ye khud apni file ki history ko code ke andar hi record karti hai. Function ka poora idea tha: jab summer-window ka data incomplete tha (kuch villages ke paas summer NDBI missing thi, monsoon cloud-cover ki wajah se), check karo ki jo villages 'valid' hain wo ek random subset hain ya kisi tarah biased. Lekin function ke andar ek `if n_invalid == 0:` branch hai jo batata hai ki jab wo missingness hi khatam ho gayi (ab sabhi 251/251 villages ke paas valid summer data hai), toh ye poora comparison ab 'moot' hai — dusra group compare karne ke liye bacha hi nahi. Function ise silently hata dene ke bajaye, ye explicitly print karta hai ki purana result (154 valid, 97 invalid ka) kahan preserved hai, taaki reproducibility bani rahe. Syntax: `core["a"].notna() & core["b"].notna()` — do boolean conditions ko '&' (AND) operator se jodna, dono True hone par hi result True. `~core["summer_valid"]` — tilde (~) NOT operator, boolean ko ulta karna (True se False, False se True). `if n_invalid == 0:` — ek conditional branch jo check karta hai ki koi missingness bachi hai ya nahi, aur us hisaab se function ka poora behavior badal deta hai.

EXAMPLE — Ek Check Ko 'Moot' Kehna Bhi Ek Honest Choice Hai

Bahut projects mein, jab koi puraana check apna matlab kho deta hai (jaise yahan, kyunki missingness hi khatam ho gayi), use chup-chaap code se hata diya jaata hai, jaise wo kabhi tha hi nahi. Yahan aisa nahi kiya gaya — function abhi bhi file mein hai, explicitly bataata hai ki ye ab moot hai, aur bataata hai kahan uska historical result dhoondha ja sakta hai. Ye chhoti si choice — delete karne ke bajaye, explicitly 'ye ab applicable nahi hai, yahan dekho purana result' likhna — exactly waisa hi hai

jaisa PDF 15 mein bhi dekha tha jab pretreatment_placebo_test.py ke bare mein baat hui thi: history ko chhupana nahi, usse transparently document karna.

5. fullyear_lights_robustness() — Ek Doosre Result Ka Wahi Stress-Test

```
# -----
# 5. same leave-one-out + randomization stress test, but for full-year lights
# DiD - this was the last still-significant control-group DiD result after
# Entry 22, but the control-list contamination fix in Entry 24/25 sent it to
# null too (see BO_Development_Log.md Entry 25). Kept here as a completed
# check on a now-null result, not as a check on something still significant.
# -----

def fullyear_lights_robustness():
    print("=" * 70)
    print("5. LEAVE-ONE-OUT + RANDOMIZATION INFERENCE, FULL-YEAR LIGHTS DiD")
    print(" (significant post-Entry-22; went null post-Entry-25 control-list fix -- checked here anyway)")
    print("=" * 70)
    df = pd.read_csv(PANEL_PATHS["full_year"])
    districts = sorted(df["district"].unique())
    rows = []
    for d in districts:
        r = run_did(df[df["district"] != d], "lights")
        r["dropped_district"] = d
        r["significant"] = r["p"] < 0.05
        rows.append(r)
        print(f" drop {d:16s} coef={r['coef']:+.5f} p={r['p']:.5f} {'YES' if r['significant'] else 'no'}")
    n_sig = sum(r["significant"] for r in rows)
    coefs = [r["coef"] for r in rows]
    print(f" -> coef range [{min(coefs):+.5f}, {max(coefs):+.5f}], significant (p<0.05) in "
          f"{n_sig}/{len(rows)} district-dropped reruns\n")
```

Purpose: Ye function bilkul function #1 (leave_one_district_out) jaisa hi kaam karta hai, lekin ek alag target par — full-year lights DiD, na ki summer wala. Comment block khud is function ki poori kahani ek jagah summarize kar deta hai: ye result Entry 22 ke baad (thodi der ke liye) 'is study ka aakhri surviving significant result' tha, lekin Entry 24/25 ke control-list contamination fix ne isse bhi null bana diya. Comment explicitly kehta hai ki ye function ab 'a completed check on a now-null result' hai, 'a check on something still significant' nahi — matlab is function ka result ab historical curiosity hai, active concern nahi. Neeche is poori dramatic history ko detail mein dekhenge. Syntax: Multi-line comment block (# se shuru, kai lines) — code ke ek poore section ka context dena, jo docstring se alag hai (docstring sirf function/module ke shuru mein hota hai, comment kahin bhi ho sakta hai). Baaki poora function structurally leave_one_district_out() jaisa hi hai, bas single outcome (lights) aur single window (full_year) par.

```

rng = np.random.RandomState(RNG_SEED)
observed = run_did(df, "lights")
obs_coef = observed["coef"]
df["_key"] = df["treatment"].astype(str) + "_" + df["village_id"].astype(str)
villages = df.drop_duplicates(subset=["_key"])[["_key", "district", "treatment"]]
perm_coefs = []
for _ in range(N_PERMUTATIONS):
    perm = villages.copy()
    for d, grp in perm.groupby("district"):
        perm.loc[grp.index, "treatment"] = rng.permutation(grp["treatment"].values)
    treat_map = dict(zip(perm["_key"], perm["treatment"]))
    df_p = df.copy()
    df_p["treatment"] = df_p["_key"].map(treat_map)
    df_p["did_term"] = df_p["treatment"] * df_p["post"]
    perm_coefs.append(run_did(df_p, "lights")["coef"])
perm_coefs = np.array(perm_coefs)
p_perm = (np.sum(np.abs(perm_coefs) >= np.abs(obs_coef)) + 1) / (len(perm_coefs) + 1)

```

Purpose: Aur ye hissa bilkul randomization_inference() ke andar ka logic hai, sirf full-year lights ke liye, single outcome par (isliye ek loop ke andar nahi, seedha ek hi baar). Same 2000-permutation, within-district shuffle, same '+1' pseudocount p-value formula. **Syntax:** Koi naya syntax nahi — ye poora block function #2 (randomization_inference) ke andar ke ek hisse ka hi repeat hai, sirf ek single outcome ke liye generalize kiye bina, seedha hardcoded.

main() — Sabhi 5 Checks Ko Ek Saath Chalana

```

def main():
    results = {
        "leave_one_district_out": leave_one_district_out(),
        "randomization_inference": randomization_inference(),
        "log_viirs": log_viirs_sensitivity(),
        "summer_missingness": summer_missingness(),
        "fullyear_lights_did_robustness": fullyear_lights_robustness(),
    }
    with open(OUT_JSON, "w") as f:
        json.dump(results, f, indent=2, default=str)
    print(f"Saved {OUT_JSON}")

```

Purpose: Sabhi paanch checks ko sequentially call karke, unke results ek single, named dictionary mein daal diye jaate hain, jo phir ek JSON file mein save hote hain. Ye woh outputs/robustness_extended_results.json hai jo hum abhi Final State mein padhenge. **Syntax:** default=str json.dump() ka ek parameter hai jo batata hai ki agar koi value directly JSON mein convert na ho sake (jaise koi custom object), toh usse plain string mein badal do, error dene ke bajaye — ek defensive touch, taaki poora save-step kisi unexpected type ki wajah se crash na ho.

Final State — extended_robustness_checks.py Ka Current, 721-Village Verdict

outputs/robustness_extended_results.json aur
outputs/leave_one_district_out_results.csv se seedha liye gaye current numbers:

Check	Outcome/Window	Result
Leave-one-out (summer)	NDBI	coef range [-0.00564, +0.00046], significant 0/14
Leave-one-out (summer)	Lights	coef range [-0.00949, +0.04238], significant 0/14
Randomization inference (summer)	NDBI	obs=-0.00275, perm mean=+0.00022, p=0.22939
Randomization inference (summer)	Lights	obs=+0.01798, perm mean=+0.00055, p=0.49325
Log1p treated-only Wilcoxon	Full-year	raw p=0.05008, log1p p=0.06772
Log1p treated-only Wilcoxon	Summer-matched	raw p=0.99999, log1p p=0.99999
Log1p control-group DiD	Full-year	raw p=0.45334, log1p p=0.61112
Log1p control-group DiD	Summer	raw p=0.64393, log1p p=0.72112
Summer missingness	core sample	MOOT — 251/251 valid, 0 invalid remaining
Leave-one-out (full-year lights)	Lights	coef range [+0.00066, +0.05818], significant 0/14
Randomization inference (full-year lights)	Lights	obs=+0.03460, perm mean=+0.00019, p=0.33533

Har ek check, bina apvaad, ab null hai — koi bhi district kisi result ko akela drive nahi kar raha, cluster-robust asymptotics ke bina bhi (yaani randomization inference se bhi) koi result significant nahi milta, log-scale transform karne se bhi koi conclusion nahi badalta, aur summer-missingness ka poora sawaal hi ab moot hai kyunki missingness khud khatam ho chuki hai. Lekin — aur yehi is file ki sabse zaroori history hai — ye hamesha se aisa nahi tha.

History — Ye File Teen Baar Apna Verdict Badal Chuki Hai

EXAMPLE — Entry 20 — File Pehli Baar Likhi Gayi, Aur Us Waqt HAR CHECK Significant Tha

Jab `extended_robustness_checks.py` pehli baar likhi gayi (Development Log Entry 20), us waqt ka summer-window control-group DiD result (154-village, Sikkim-excluded sample par based) genuinely significant tha, aur is file ke sabhi checks ne ise support kiya: leave-one-district-out mein dono outcomes (NDBI, lights) 14/14 reruns mein significant rahe (NDBI coefficient range +0.0240 se +0.0359, lights ka +0.1117 se +0.1731); randomization inference mein bhi dono outcomes ka observed coefficient apne 2000-permutation null se bahar tha (dono $p=0.0005$); log1p transform karne se result 'strengthen' hua (raw $p=0.0044$ se log1p $p=0.0003$ tak); aur summer-missingness check ne dikhaya ki 154 valid villages ek random subset nahi the — state composition mein alag ($\chi^2=71.42$, $p\approx 3.1\times 10^{-16}$) aur baseline NDBI mein bhi alag (Mann-Whitney $p=0.00063$). Us waqt, ye poora set of checks is study ka sabse heavily-stress-tested, sabse convincing result tha.

EXAMPLE — Entry 21 — Ek Chhota Sa Shaq, Jo Bahut Bada Nikla

Development Log Entry 21 mein ek external review ne ek sawaal poochha jo khud galat tha apne specific claim mein, lekin sahi tarah ki jaanch ki taraf ishara kar gaya. Us jaanch mein pata chala ki `extract_satellite_data.py` (asli two-point extraction) aur `extract_multiyear_satellite_data.py` (teen-point extraction, baad mein banaya gaya), dono hi bilkul identical query chalate hain summer-window 2021/2025 ke liye — same collection, same cloud-mask, same date range. Lekin unke results match nahi karte: two-point extraction mein Sikkim ke sabhi 31 villages ka before/after image-count 0 hai (koi scene nahi mila), jabki teen-point extraction (jo baad mein chali) usi Sikkim ke villages ke liye 24 aur 31 scenes dhoondh leti hai, bilkul same date-range ke liye. Ye koi code-bug nahi tha — ye Sentinel-2 ka apna satellite-archive tha, jo do alag extraction-dates ke beech mein naye scenes 'backfill' kar chuka tha. Matlab: 154-village sample ka poora foundation — 'Sikkim summer mein missing hai' — ek permanent fact nahi tha, balki us din extraction chalane ka ek time-sensitive artifact tha.

EXAMPLE — Entry 22 — Poora Result Palat Gaya, Jaise Hi Sikkim Wapas Aaya

Development Log Entry 22 mein poori summer-window extraction dobara, fresh, same-day (treated aur control dono ek hi din) chalayi gayi. Sikkim ab include ho gaya (258/258 treated, 735/735 control, dono complete). Aur jaise hi ye hua, is file ka har ek check apna verdict palat gaya: H1 treated-only NDBI Wilcoxon ($n=154$, $p<0.000001$, significant) se ($n=251$, $p=0.4358$, non-significant) ban gaya; H4 NDBI control-group DiD (coef=+0.0322, $p=0.00033$) se (coef=-0.0038, $p=0.44701$ — sign hi flip ho gaya) ban gaya; H4 lights DiD (coef=+0.1413, $p=0.0044$) se (coef=+0.0374, $p=0.36553$) ban gaya; leave-one-district-out 14/14-significant se 0/14-significant ban gaya; randomization inference $p=0.0005/0.0005$ se $p=0.115/0.094$ ban gaya; log1p-strengthening (raw $p=0.0044\rightarrow$ log1p $p=0.0003$) ab dono null reh gaye (raw $p=0.366\rightarrow$ log1p $p=0.426$). Interesting taur par, ek naya result significant nikal aaya jahan pehle stable null tha — H3 NDBI border-proximity (Spearman $\rho=0.038$,

$p=0.641$ se $\rho=0.291$, $p=2.6\times 10^{-6}$ tak) — yehi wo result hai jo aage PDF 13 aur PDF 16 mein poora dekha gaya. Aur baseline (2021) balance bhi behtar ho gaya — treated aur control ab statistically indistinguishable the summer-window mein (pehle $p=0.00073/0.0165$, ab $p=0.478/0.354$).

EXAMPLE — Entry 24/25 — Dusra Data-Fix, Aur Aakhri Bacha Hua Significant Result Bhi Chala Gaya

Entry 22 ke baad, ek result bacha reh gaya tha jo ab bhi significant tha — full-year lights DiD (coef=+0.1653, $p=0.0364$). Ye function #5 (fullyear_lights_robustness) exactly isi result ko stress-test karne ke liye likha gaya tha, aur us waqt ye check bhi is result ko support karta tha — leave-one-out mein 10/14 reruns significant, randomization inference $p=0.0155$. Lekin Entry 23-24-25 ki jaanch mein control-village list mein hi ek asli contamination problem mili — 20 control villages physically wahi settlements the jo already treated-side mein the (sirf naam-variant ke saath), aur 3 aur officially-VVP-listed-lekin-ungeocoded treated villages, apne asli naam se, galti se control list mein reh gaye the. Jab ye fix hua (735 se 732, fir baad mein 721 tak district-verification ke saath), ye aakhri significant result bhi gir gaya: full-year lights DiD coef=+0.1653/ $p=0.0364$ se coef=+0.0505/ $p=0.30946$ ban gaya; leave-one-out 10/14-significant se 0/14-significant ban gaya (jo hum abhi is file ke current output mein dekh rahe hain); randomization inference $p=0.0155$ se $p=0.12994$ (aur baad mein 721-list ke saath $p=0.33533$, jo hum abhi upar dekha) ban gaya; log1p bhi dono taraf null ho gaya. Development Log khud is moment ko explicitly is tarah describe karta hai: 'a result that looked like it survived every check, until the actual data problem behind it was fixed rather than worked around.'

EXAMPLE — Sabse Zaroori Sabak — Ye Checks Khud Kabhi Galat Nahi The

Is poori dramatic history ka sabse important takeaway ye NAHI hai ki leave-one-out ya randomization inference 'kaam nahi karte.' In dono checks ka apna implementation shuru se hi sahi tha — docstring mein khud likha hai ki inhe pehle did_model.py ke committed numbers ke against verify kiya gaya, 5+ significant figures tak. Ye checks har baar bilkul sahi kaam kar rahe the — jo bhi data unhe diya gaya, uske against wo faithfully, honestly report kar rahe the ki 'is data ke hisaab se, ye result robust hai ya nahi.' Jab data khud galat tha (Sikkim missing, ya control-list contaminated), checks ne bhi ek galat, lekin us galat-data ke hisaab se consistent, verdict diya. Jab data theek hua, checks ka verdict bhi apne aap sahi ho gaya. Ye is poore project ka ek central lesson hai: robustness-checks apni jagah zaroori hain, lekin wo underlying data ki galtiyan ko theek nahi kar sakte — sirf ye confirm kar sakte hain ki jo bhi data hai, uske against result kitna stable hai. Data ki quality khud, alag se, verify honi hi padti hai.

Part B — geocoding_selection_bias_check.py

Docstring Aur Imports

"""

Checks whether geocoding success (which of the officially-listed VVP-I priority villages made it into the geocoded, core-sample dataset) correlates with village size (population/households) or with district -- i.e. whether the 251-village core sample is a biased subset of the 559 officially listed priority villages across the three core states, not just an incomplete one.

Uses only files already in this repository (data/raw/*_vvp_villages.csv, data/processed/border_optics_master_villages_with_distance.csv) -- no new data collection. Name matching reuses select_control_villages.py's own normalize_name() so this check is consistent with how the rest of the pipeline already does village-name matching.

Adds one new angle (district composition) alongside the population/household check BO_Research_Paper.md Section 6.5 already reported.

Run: python src/analysis/geocoding_selection_bias_check.py

"""

```
import json
```

```
import pandas as pd
from scipy import stats
```

Purpose: Ye docstring ek bahut zaroori distinction khinchta hai: 'incomplete' hona (matlab, sirf kam villages geocode hue) ek baat hai; 'biased' hona (matlab, jo geocode hue wo systematically kisi khaas tarah ke villages the — jaise bade villages, ya kisi khaas district ke) ek bilkul alag, zyada serious baat hai. Ye script check karta hai ki 251-village core sample, 559 officially-listed priority villages (teeno core states milakar) ka ek unbiased subset hai ya nahi. Docstring ye bhi bataata hai ki koi nayi data collection nahi ki gayi — sirf already-existing files use hue hain, aur name-matching ke liye select_control_villages.py (PDF 7) ka apna normalize_name() function reuse kiya gaya hai, taaki poore pipeline mein village-naam match karne ka tareeka consistent rahe. Syntax: import json, pandas as pd, from scipy import stats — familiar imports. Koi naya syntax nahi is baar, docstring khud sabse zaroori hissa hai.

Paths

```
MASTER_PATH = "data/processed/border_optics_master_villages_with_distance.csv"
RAW_PATHS = {
    "Arunachal Pradesh": "data/raw/arunachal_pradesh_vvp_villages.csv",
    "Sikkim": "data/raw/sikkim_vvp_villages.csv",
    "Uttarakhand": "data/raw/uttarakhand_vvp_villages.csv",
}
OUT_JSON = "outputs/geocoding_selection_bias_check.json"
OUT_CSV = "outputs/geocoding_selection_bias_district_summary.csv"
```

Purpose: MASTER_PATH — wahi master file jismein sabhi geocode-hue villages (treated aur core-sample dono) apne coordinates aur distance-to-border ke saath hain. RAW_PATHS — teen core states ke apne, raw, officially-published VVP-I priority-village lists — ye woh 'ground truth' lists hain jinke against geocoding-success check hoga. Syntax: Dictionary literal, state-naam se file-path map karta hua — is series mein bahut baar dekha gaya pattern.

normalize_name() — select_control_villages.py Se Wahi Function, Reused

```
def normalize_name(name):
    # identical to select_control_villages.py's normalize_name(), duplicated
    # here rather than imported to keep this a standalone, one-file check
    import unicodedata
    name = unicodedata.normalize("NFKD", str(name)).encode("ascii", "ignore").decode()
    return " ".join(name.lower().split())
```

Purpose: Ye function PDF 7 mein select_control_villages.py mein poore detail se seekha gaya tha — village-naam ko ek consistent, comparable form mein laata hai: accented ya special characters hataana (jaise koi naam jisme French-jaisa accent-mark ho, ya ek hi village do alag OSM-tags mein thoda alag spelling ke saath likha ho), sabkuch lowercase karna, aur extra spaces hata dena. Comment khud ek zaroori design-choice explain karta hai: iss function ko import karne ke bajaye (jo cleaner lagta), yahan dobara likha gaya hai, taaki ye script apne aap mein standalone rahe — kisi doosri file par depend na kare sirf ek chhoti-si helper function ke liye. Ye ek trade-off hai: thoda code-duplication, lekin zyada independence. Syntax: import unicodedata (function ke andar hi, top-level par nahi) — Python mein function ke andar bhi import kiya ja sakta hai, jab wo import sirf usi function ke liye zaroori ho. unicodedata.normalize("NFKD", ...) — PDF 7 mein poora dekha tha, accented characters ko unke base-letter + accent-mark mein todta hai. .encode("ascii", "ignore").decode() — sirf plain ASCII characters rakhna, baaki (accent marks) hata dena. " ".join(name.lower().split()) — sabhi extra whitespace normalize karke ek single space se replace karna.

main() — Geocoding Coverage Check

```
def main():
    master = pd.read_csv(MASTER_PATH)
    master_core = master[master["is_core_sample"] == True].copy()
    master_core["village_norm"] = master_core["village"].apply(normalize_name)
    geocoded_names = set(master_core["village_norm"])

    all_rows = []
    state_summary = {}
    for state, path in RAW_PATHS.items():
```

```

df = pd.read_csv(path)
df["village_norm"] = df["Habitation"].apply(normalize_name)
df["state"] = state
df["geocoded"] = df["village_norm"].isin(geocoded_names)
n_total = len(df)
n_geo = int(df["geocoded"].sum())
state_summary[state] = {
    "n_official_villages": n_total,
    "n_geocoded": n_geo,
    "n_ungeocoded": n_total - n_geo,
    "geocoded_pct": round(n_geo / n_total * 100, 1),
}
all_rows.append(df)
combined = pd.concat(all_rows, ignore_index=True)

```

Purpose: Pehle, master file se sirf core-sample villages ka normalized naam nikaal kar ek 'set' banaya jaata hai — set isliye kyunki hume sirf ye check karna hai ki koi naam ismein 'hai ya nahi,' fast lookup ke liye (list ke bajaye set use karna, .isin() jaisi checks ko bahut fast banata hai). Fir teeno states ki raw official lists ko baari-baari padho, unka apna normalized naam banao, aur check karo ki wo naam geocoded_names set mein hai ya nahi — agar hai, matlab wo village successfully core-sample mein geocode ho gaya; agar nahi, toh wo officially listed toh tha, lekin geocoding pipeline usse capture nahi kar payi. Har state ka apna coverage-percentage nikaal ke ek summary dictionary banaya jaata hai. Syntax: set(collection) — ek Python data-structure jo sirf unique values rakhta hai aur membership-check (x in set) bahut fast karta hai, list ke comparatively. df["col"].apply(function) — poore column ke har element par ek function call karna, PDF 7 mein bhi dekha. df["col"].isin(set) — har row check karta hai ki uski value us set/collection mein hai ya nahi, True/False ka boolean column deta hai. pd.concat(list_of_dfs, ignore_index=True) — kai dataframes ko ek ke neeche ek jodkar ek bada dataframe banana.

```

ap = combined[combined["state"] == "Arunachal Pradesh"].copy()
size_results = {}
for col in ["Population", "Households"]:
    valid = ap.dropna(subset=[col])
    geo = valid[valid["geocoded"] == True][col]
    ungeo = valid[valid["geocoded"] == False][col]
    u_stat, p = stats.mannwhitneyu(geo, ungeo, alternative="two-sided")
    size_results[col] = {
        "geocoded_mean": float(geo.mean()), "geocoded_n": int(len(geo)),
        "ungeocoded_mean": float(ungeo.mean()), "ungeocoded_n": int(len(ungeo)),
        "mannwhitney_p": float(p),
    }

```

Purpose: Ye Section 6.5 ka pehla, purana angle hai — kya geocoded villages, size (Population, Households) mein ungeocoded villages se alag hain. Comment mein

pehle bhi dekha hai ki ye sirf Arunachal Pradesh ke liye possible hai, kyunki Sikkim aur Uttarakhand ki raw lists mein ye columns hi nahi hain. Har size-metric ke liye, geocoded aur ungeocoded, dono groups ki values nikaal ke Mann-Whitney U test (PDF 12 mein pehli baar dekha — do independent groups ki raw distributions compare karna) chalaya jaata hai. Syntax: `combined["state"] == "..."` — ek specific state ki rows filter karna. `geo = valid[valid["geocoded"] == True][col]` — do-step filtering: pehle `geocoded==True` wali rows lo, fir unka col column nikaalo. `stats.mannwhitneyu(a, b, alternative="two-sided")` — PDF 12 mein bhi dekha gaya function, yahan two-sided (kisi bhi taraf farak check karna, sirf ek direction nahi, kyunki yahan koi specific hypothesis nahi hai ki kaunsa bada hoga).

District Composition Check — Naya Angle, Jo Section 6.5 Mein Pehle Nahi Tha

```
district_summary = combined.groupby(["state", "District"]).agg(
    total=("geocoded", "size"), geocoded=("geocoded", "sum")
).reset_index()
district_summary["geocoded"] = district_summary["geocoded"].astype(int)
district_summary["rate_pct"] = (district_summary["geocoded"] /
district_summary["total"] * 100).round(1)
district_summary.to_csv(OUT_CSV, index=False)

district_chi2 = {}
for state in RAW_PATHS:
    sub = combined[combined["state"] == state]
    ct = pd.crosstab(sub["District"], sub["geocoded"])
    chi2, p, dof, _ = stats.chi2_contingency(ct)
    district_chi2[state] = {
        "chi2": float(chi2), "dof": int(dof), "p": float(p), "n_districts": int(len(ct)),
    }
```

Purpose: Ye is script ka naya, extra angle hai — sirf size nahi, balki district ke hisaab se bhi geocoding-rate compare karna. `groupby(["state", "District"]).agg(...)` ek saath do columns nikaalta hai har state-district combination ke liye: total (kitne officially-listed villages) aur geocoded (kitne unme se geocode hue, sum se, kyunki `True=1` count hota hai). Fir har state ke liye alag-alag ek chi-square contingency test chalaya jaata hai — 'kya geocoding-success district ke hisaab se evenly-spread hai, ya kuch districts mein zyada, kuch mein kam hai, jitna chance se expect hota.' Syntax: `groupby([...]).agg(new_name=("col", "function"))` — pandas ka 'named aggregation' syntax, ek saath multiple summary-statistics banana aur unhe naye naam dena, ek hi step mein. `.reset_index()` — `groupby` ke baad group-columns ko wapas normal columns bana dena (warna wo dataframe ka index ban jaate hain). `pd.crosstab(a, b)` — do categorical columns ke beech ek contingency table (cross-tabulation) banana — har combination ki count. `stats.chi2_contingency(table)` — PDF 15 mein bhi dekha gaya, ek contingency table par chi-square independence test, batata hai ki do

categorical variables (yahan: district, geocoded-ya-nahi) associated hain ya nahi.

Final State — geocoding_selection_bias_check.py Ka Current Verdict

outputs/geocoding_selection_bias_check.json aur
outputs/geocoding_selection_bias_district_summary.csv se seedha liye gaye numbers:

State	Official Villages	Geocoded	Ungeocoded	Coverage %
Arunachal Pradesh	455	186	269	40.9%
Sikkim	46	31	15	67.4%
Uttarakhand	51	34	17	66.7%

Poore teeno core states milakar, 559 officially-listed VVP-I priority villages mein se sirf 251 hi geocode ho paaye — matlab 44.9% overall coverage. Arunachal Pradesh, jo sabse bada aur sabse remote state hai, sabse kam coverage (40.9%) dikhaata hai.

Check	Result
Population (Arunachal only)	geocoded mean=135.0 (n=186), ungeocoded mean=145.7 (n=268), MW p=0.30950
Households (Arunachal only)	geocoded mean=25.84 (n=186), ungeocoded mean=28.85 (n=269), MW p=0.53980
District chi-square (Arunachal Pradesh)	chi2=139.16, df=10, p=6.25e-25, 11 districts
District chi-square (Sikkim)	chi2=6.01, df=1, p=0.01425, 2 districts
District chi-square (Uttarakhand)	chi2=6.24, df=2, p=0.04420, 3 districts

Village-size (Population, Households) ke hisaab se koi significant bias nahi mila — geocoded aur ungeocoded villages, average size mein statistically indistinguishable hain (dono p-values 0.05 se kaafi upar). Lekin district-level par, teeno states mein, geocoding-success bilkul bhi evenly-spread nahi hai — sabhi teeno chi-square tests significant hain, Arunachal Pradesh ka toh khaas taur par extreme (p ek astronomically chhota number hai, 6.25×10^{-25}).

EXAMPLE — Ye Abstract Numbers Nahi Hain — Ek Concrete Example Dekho

outputs/geocoding_selection_bias_district_summary.csv seedha dikhaata hai ki ye district-level farak kitna extreme hai: Shiyomi district (Arunachal Pradesh) mein 30

officially-listed villages hain, aur unme se EK bhi geocode nahi hua — 0.0% coverage. Tawang, jo official list ka sabse bada district hai (155 villages, poore study ka lagbhag ek-tihaai), sirf 16.8% (26 villages) geocode hua. Doosri taraf, Kurung Kumey ne 83.3% (70 out of 84) geocode kiya, aur Uttarkashi (Uttarakhand) ne poora 100% (10 out of 10). Sikkim ka East district bhi 0% (0 out of 4) hai, jabki North district 73.8% (31 out of 42) hai. Ye bilkul random nahi lag raha — jaise koi districts jinke paas better roads, connectivity, ya OSM-data-density thi, unki geocoding zyada successful rahi, jabki sabse remote, sabse difficult-to-reach districts (jaise Shiyomi, Tawang ka bada hissa) peeche reh gaye.

EXAMPLE — Kyun Ye Study Ke Apne H3 Hypothesis Ke Liye Extra Zaroori Hai

Ye district-imbalance sirf ek generic 'sample thoda incomplete hai' wala disclaimer nahi hai — ye is study ke apne H3 hypothesis (border-proximity) ke liye khaas taur par relevant hai. Tawang aur Shiyomi, jinki geocoding-rate sabse kam hai (16.8% aur 0%), Arunachal Pradesh ke sabse remote, sabse border-adjacent districts mein se hain — exactly wahi tarah ki jagah jo H3 ka test sabse zyada interested hai. Agar border ke sabse paas ke villages hi systematically geocoding se chhoot rahe hain, toh 251-village core sample border-proximity ke hisaab se bhi thoda biased ho sakta hai — na sirf district ke hisaab se. Ye is study ka Development Log khud explicitly is tarah likhta hai, aur ise 'ek open limitation' ki tarah disclose karta hai, jise fix karne ke liye 301 currently-ungeocoded officially-listed villages ka ek fresh geocoding-pass chahiye hoga — jo abhi is project ke scope se bahar hai.

History — `geocoding_selection_bias_check.py` Kahan Se Aayi

EXAMPLE — Entry 36 — Ek External Review Ka Suggestion, Jo Sahi Tha Lekin Modify Karna Pada

Development Log Entry 36 mein ek external reviewer (Anara) ne suggest kiya tha ki geocoded aur ungeocoded villages ko altitude aur border-distance ke hisaab se compare kiya jaaye. Lekin jaanch karne par pata chala ki ye comparison genuinely computable hi nahi hai — ungeocoded villages, by definition, ke paas koi coordinates hi nahi hain (unhe kabhi geocode nahi kiya gaya), isliye unki altitude ya border-distance compute karne ke liye kuch hai hi nahi. Ye check karke, guess kiye bina, confirm kiya gaya raw official village lists ke against. Jo comparison actually available tha — District/Block/Population — wahi banaya gaya, aur District-composition wala naya angle jodkar, jo pehle se maujood Population/Households check (Section 6.5) ke saath combine kiya gaya. Result: teeno core states mein geocoding success district ke hisaab se bilkul bhi uniform nahi hai (Arunachal Pradesh $\chi^2=139.16$, $df=10$, $p=6.25 \times 10^{-25}$; Sikkim $\chi^2=6.01$, $p=0.0143$; Uttarakhand $\chi^2=6.24$, $p=0.0442$) — is baat ko upar detail mein dekh chuke hain.

EXAMPLE — Ek Chhota Course-Correction Bhi Isi Entry Mein Record Hai

Entry 36 mein ek aur honest moment record hai jo dhyaan dene layak hai: is round mein researcher ne teen extra standalone draft files bhi ban wa li thi Anara ke baaki suggestions ke liye, lekin beech mein khud realize kiya ki unme se ek (equivalence-framing draft) ke liye kabhi explicitly nahi poocha gaya tha. Course sudhaara gaya — bina defensive hue — aur baaki kaam seedha BO_Research_Paper.md mein hi edit karke kiya gaya, alag draft files banane ke bajaye. Ye chhota sa moment is poore project ki ek badi pattern dikhaata hai: jab bhi apne aap se zyada scope-creep hoti dikhe, use turant pakadkar sudhaarna, chhupaana nahi.

Part C — power_analysis.py

H1 aur H4, dono hi null aaye hain — matlab koi statistically significant effect nahi mila. Lekin ' $p > 0.05$ ' sirf itna kehta hai ki 'jo bhi effect hai (agar hai), wo itna bada nahi tha ki humare data mein clearly dikh sake.' Ye 'no effect' aur 'effect jo humara design detect nahi kar sakta tha' — dono mein farak nahi karta. power_analysis.py exactly isi gap ko bharta hai: 'Minimum Detectable Effect' (MDE) nikaal kar, jo bataata hai ki humara sample-size aur design, kitna CHHOTA effect bhi reliably pakad sakta tha, agar wo waha hota.

Docstring Aur Imports

```
"""
Minimum Detectable Effect for H1 and H4, since both came back null and I
want to report "no effect bigger than X is detectable" not just "p>0.05".
H1 uses paired-design MDE (df=n-1), H4 uses the cluster-robust SE from
did_model.py with df = 14 districts - 1 (Cameron & Miller 2015 for thin clusters).

Usage:
    python3 src/analysis/power_analysis.py
"""

import json

import numpy as np
import pandas as pd
from scipy import stats
```

Purpose: Docstring seedha is file ka poora maqsad bata deta hai — sirf ' $p>0.05$ ' kehne ke bajaye, 'is design se X se bada koi bhi effect detect ho jaata, isliye agar koi effect hai, wo X se chhota hona chahiye' jaisi zyada informative statement dena. Do alag MDE formulas ka istemal hota hai: H1 ke liye paired-design formula ($df = n-1$, kyunki har village ka apna before/after pair hai), aur H4 ke liye already-fitted cluster-robust SE se, $df = 14 \text{ districts} - 1 = 13$ (Cameron & Miller 2015 ka reference, thin-cluster situations ke liye ek established statistical practice), PDF 14 mein wild cluster bootstrap ke

context mein bhi jo '14 clusters bahut kam hain' wali baat dekhi thi, usi thin-cluster concern se related hai. Syntax: Standard imports — koi naya syntax nahi.

Constants

```
ALPHA = 0.05 # nominal alpha; sidedness is set per-test below to match the
              # actual inferential test each MDE is calibrated against
POWER = 0.80
N_DISTRICTS = 14

TREATED_PATHS = {
    "full_year": "data/processed/border_optics_village_results_analyzed.csv",
    "summer": "data/processed/border_optics_village_results_summer_analyzed.csv",
}
DID_SUMMARY_PATHS = {
    "full_year": "data/processed/border_optics_did_summary_fullyear.json",
    "summer": "data/processed/border_optics_did_summary_summer.json",
}
OUT_PATH = "data/processed/border_optics_power_analysis.json"
```

Purpose: ALPHA=0.05 (standard significance threshold) aur POWER=0.80 (standard convention — 80% chance ki agar effect genuinely exist karta hai, humara test use pakad legi) — dono statistics mein widely-accepted default values hain, khud invent nahi kiye gaye. Comment ALPHA ke saath hi ek zaroori nuance flag karta hai (aage detail mein dekhenge): sidedness (one-sided ya two-sided) har test ke liye alag set hoti hai, us test ki apni actual nature ke hisaab se. Syntax: Modules-level constants, is series mein bahut baar dekha gaya pattern. Dictionary literals, file-paths store karne ke liye.

mde_multiplier() — Ek Statistical Formula, Do Sidedness Ke Liye

```
def mde_multiplier(df, alternative="two-sided"):
    # standard MDE formula, using t-quantiles instead of z since df is thin here.
    # alternative="two-sided" matches H4's cluster-robust DiD test (did_model.py uses
    # statsmodels' default two-sided p-values). alternative="one-sided" matches H1's
    # actual test (analyze_results.py / did_model.py call stats.wilcoxon(...,
    # alternative="greater")) -- using two-sided alpha there would understate power
    # relative to the test actually run.
    t_alpha = stats.t.ppf(1 - ALPHA / 2, df) if alternative == "two-sided" else stats.t.ppf(1 -
    ALPHA, df)
    t_power = stats.t.ppf(POWER, df)
    return t_alpha + t_power
```

Purpose: Ye function MDE (Minimum Detectable Effect) formula ka core hissa hai. Standard MDE formula hoti hai: $MDE = (t_alpha + t_power) \times (\text{standard error})$. Ye function sirf pehla hissa ($t_alpha + t_power$) compute karta hai — ek 'multiplier' jo baad mein standard-error se multiply hoga. Comment ek bahut zaroori design-choice detail se explain karta hai: H1 ka asli test (Wilcoxon, alternative='greater') one-sided

hai — hum sirf 'increase hua kya' poochh rahe hain. H4 ka asli test (statsmodels' default cluster-robust p-values) two-sided hai — 'kisi bhi taraf farak hua kya.' MDE calibration ko is asli test ke sidedness se match karna zaroori hai, warna power ka number galat direction mein bias ho jaata — jaise agar H1 ke liye galti se two-sided threshold use kiya jaata, toh MDE zyada bada dikhta (matlab, power ko understate karta), jabki asli test one-sided hone ki wajah se genuinely usse chhote effects bhi pakad sakta hai. Syntax: stats.t.ppf(q, df) — t-distribution ka 'percent point function' (inverse CDF), ek probability (q) diya jaaye toh us probability tak ka t-value deta hai — jaise stats.t.ppf(0.95, df) wo t-value dega jiske neeche 95% probability hai. df yahan thin hai (jaise H4 ke liye sirf 13), isliye t-distribution use hui hai, normal (z) distribution nahi — t-distribution ki fat tails chhote-sample uncertainty ko better account karti hain. Conditional expression (ternary) ek line mein: value_if_true if condition else value_if_false.

EXAMPLE — MDE Formula Ka Intuition — Ek Metal Detector Ki Sensitivity

MDE ka formula $(t_{\alpha} + t_{\text{power}}) \times \text{SE}$ samajhne ka ek tareeka: t_{α} wo threshold hai jise paar karna zaroori hai 'significant' kehlaane ke liye (jitna sakht alpha, utna bada t_{α}). t_{power} wo extra 'buffer' hai jo add karna padta hai taaki agar effect genuinely utna bada ho, toh 80% chance rahe usse pakadne ka (sirf 50% nahi, jahan theek threshold ke aas-paas coin-flip jaisa ho jaata). In dono ko jodkar, standard error se multiply karne par, ek number milta hai jo batata hai: 'agar asli effect ism se chhota hai, toh humare design ka 80% chance nahi hai use significant find karne ka.' Bilkul jaise ek metal detector ki 'sensitivity rating' bataati hai ki wo kitne chhote sikke ko reliably beep karega — usse chhoti cheez shayad kabhi-kabhi pakdega, kabhi nahi, guaranteed nahi.

h1_mde() — Treated-Only, Paired-Design MDE

```
def h1_mde(window):
    """Primary estimand: paired treated-only before/after MDE, core sample.
    One-sided, matching the actual H1 test (alternative='greater')."""
    df = pd.read_csv(TREATED_PATHS[window])
    core = df[df["is_core_sample"] == True]
    baseline_lights = core["lights_before"].mean()
    rows = []
    for outcome in ["ndbi_change", "lights_change"]:
        s = core[outcome].dropna()
        n = len(s)
        sd = s.std()
        dof = n - 1
        mult = mde_multiplier(dof, alternative="one-sided")
        mde = mult * sd / np.sqrt(n)
        row = {
            "test": "H1_treated_only", "window": window,
            "outcome": outcome.replace("_change", ""),
            "n": n, "sd_of_change": sd, "df": dof,
```

```

    "mde": mde, "mde_as_cohens_d": mde / sd,
}
if outcome == "lights_change":
    row["mde_pct_of_baseline"] = 100 * mde / baseline_lights
    rows.append(row)
return rows

```

Purpose: H1 ka MDE, treated-only before/after paired-design ke liye. Har outcome (ndbi_change, lights_change) ke liye: change ki values ka standard deviation (sd) nikaalo, degrees of freedom = n-1 (paired-test ka standard df, kyunki har village apna khud ka before/after control hai), mde_multiplier() se one-sided multiplier lo, aur formula $MDE = multiplier \times SD / \sqrt{n}$ apply karo (ye standard-error ka apna formula hai, SD ko $\sqrt{n}$ se divide karke). Result mein mde_as_cohens_d bhi hai — MDE ko SD se divide karke, ek 'standardized effect size' banana, jo alag-alag scale wale outcomes (NDBI 0-1 range, lights bahut bada range) ko compare karne layak banata hai. Lights ke liye ek extra metric bhi hai — MDE ko baseline lights-level ka percentage batana, jo zyada intuitively samajh aane wala number hai ('kitna % change detect ho sakta'). Syntax: s.std() — pandas Series ka standard deviation, built-in method. mde / sd — Cohen's d ka formula, effect-size ko SD ke units mein express karna. if outcome == "lights_change": row["..."] = ... — conditionally, dictionary mein ek extra key sirf ek specific case ke liye jodna.

EXAMPLE — Ek Interesting Fact — Cohen's d Yahan Sabhi 4 Combinations Mein EK JAISA Hai

Agar tum current numbers dekho (aage Final State mein), tumhe pata chalega ki H1 ka mde_as_cohens_d — 0.15742195764112354 — full-year NDBI, full-year lights, summer NDBI, summer lights, sabhi CHAARON combinations mein EXACTLY same hai, decimal-tak. Ye koi coincidence nahi hai, balki formula ka apna structure hai: $mde_as_cohens_d = mde / sd = (multiplier \times sd / \sqrt{n}) / sd = multiplier / \sqrt{n}$ — sd khud cancel ho jaata hai! Cohen's d sirf n (sample size, jo chaaron combinations mein same 251 hai) aur df (jo bhi same hai) par depend karta hai, outcome ki apni scale ya spread par bilkul nahi. Yehi wajah hai ki Cohen's d ko 'standardized' effect-size kehte hain — ye ek design property hai (kitna bada sample, kitna sakht test), na ki kisi specific outcome ki property.

h4_mde() — Control-Group DiD MDE, Already-Fitted SE Se

```

def h4_mde(window):
    """Control-group DiD MDE, from the already-fitted cluster-robust SE.
    Two-sided, matching the actual H4 test (did_model.py's cluster-robust OLS p-values
    and the two-sided Mann-Whitney check)."""
    with open(DID_SUMMARY_PATHS[window]) as f:
        summ = json.load(f)
    dof = N_DISTRICTS - 1
    mult = mde_multiplier(dof, alternative="two-sided")

```

```

rows = []
for r in summ["did"]:
    se = r["did_se"]
    mde = mult * se
    rows.append({
        "test": "H4_control_group_DiD", "window": window,
        "outcome": r["outcome"],
        "n_obs": r["n_obs"], "se": se, "df": dof,
        "mde": mde, "observed_coef": r["did_coef"], "observed_p": r["did_p"],
        "observed_as_fraction_of_mde": abs(r["did_coef"]) / mde,
    })
return rows

```

Purpose: H4 ke liye tareeka thoda alag hai — kyunki `did_model.py` ne already cluster-robust SE fit kiya hua hai (already-committed JSON mein), yahan usse dobara compute karne ki zaroorat nahi, seedha already-fitted SE uthaakar $MDE = multiplier \times SE$ formula laga di jaati hai (ab $\sqrt{n}$ se divide nahi karna, kyunki SE khud already 'per-coefficient' scale par hai, raw SD nahi). $df = 14 \text{ districts} - 1 = 13$, two-sided multiplier (H4 ka asli test two-sided hai). Sabse interesting field hai `observed_as_fraction_of_mde` — asli observed coefficient (chahe kitna bhi ho, chahe non-significant) ko MDE se divide karke ye batata hai ki 'jo humein mila, wo humare detectable-threshold ka kitna percent tha.' Ye ek bahut zyada informative number hai sirf p-value se — agar observed coefficient MDE ka sirf 15% hai, toh clearly humara result 'chhote-se-chhota sambhav effect bhi threshold se bahut door hai,' non-significant hone ki wajah confidently 'no meaningful effect' hai, 'design bahut weak thi' nahi. Syntax: with `open(path)` as `f`: `json.load(f)` — file open karke JSON parse karna, ek line mein. `summ["did"]` — JSON file ke andar ek specific key ki value nikaalna, jo yahan ek list of dictionaries hai (har outcome ka apna record). `abs(value)` — absolute value, negative coefficient ko bhi positive treat karna, kyunki hume sirf 'magnitude, MDE ke against kitni bhi direction mein' chahiye.

main() — Sabhi 8 Combinations Print Aur Save Karna

```

def main():
    all_rows = []
    for window in ["full_year", "summer"]:
        all_rows.extend(h1_mde(window))
        all_rows.extend(h4_mde(window))

    print(f"\nMinimum Detectable Effect -- alpha={ALPHA}, power={POWER}")
    print("(H1 one-sided, matching alternative='greater'; H4 two-sided, matching the cluster-robust DiD test)\n")
    for r in all_rows:
        if r["test"] == "H1_treated_only":
            extra = f" ({r['mde_pct_of_baseline']:.1f}% of baseline)" if "mde_pct_of_baseline" in r else ""

```

```

print(f"[H1 {r['window']:>9s} {r['outcome']:7s}] n={r['n']:3d} "
      f"MDE = {r['mde']:.5f} (Cohen's d = {r['mde_as_cohens_d']:.3f}){extra}")
else:
    print(f"[H4 {r['window']:>9s} {r['outcome']:7s}] "
          f"MDE = {r['mde']:.5f} observed coef = {r['observed_coef']:+.5f} "
          f"(observed effect is {r['observed_as_fraction_of_mde']*100:.0f}% of the
detectable threshold)")

with open(OUT_PATH, "w") as f:
    json.dump(all_rows, f, indent=2)
print(f"\nSaved {OUT_PATH}")

```

Purpose: Dono windows (full_year, summer) ke liye, H1 aur H4, dono ka MDE nikaal kar ek single list mein daala jaata hai — total 8 rows (2 windows × 2 tests × 2 outcomes ka combination). Print statements har row-type ke liye alag format use karte hain (H1 ka apna Cohen's-d-included format, H4 ka apna fraction-of-MDE format), aur poori list ek JSON file mein save ho jaati hai. Syntax: 'in' operator dictionary ke saath — "mde_pct_of_baseline" in r — check karta hai ki ek key dictionary mein exist karti hai ya nahi, conditional formatting ke liye. f"{r['window']:>9s}" — right-aligned string formatting, width 9. Nested f-string ke andar dictionary-access aur conditional, is poore series mein bahut baar dekha gaya pattern ab tak.

Final State — power_analysis.py Ka Current Verdict

data/processed/border_optics_power_analysis.json se seedha liye gaye current 8 numbers (721-village control list ke against):

Test	Window	Outcome	n / SE	MDE	Observed / Cohen's d
H1 treated-only	Full-year	NDBI	n=251	0.008017	Cohen's d=0.157
H1 treated-only	Full-year	Lights	n=251	0.090856	d=0.157 (21.2% of baseline)
H1 treated-only	Summer	NDBI	n=251	0.005308	Cohen's d=0.157
H1 treated-only	Summer	Lights	n=251	0.070951	d=0.157 (17.7% of baseline)

H4 control-DiD	Full-year	NDBI	SE=0.006330	0.019182	obs=+0.00668 (34.8% of MDE)
H4 control-DiD	Full-year	Lights	SE=0.046139	0.139825	obs=+0.03460 (24.7% of MDE)
H4 control-DiD	Summer	NDBI	SE=0.004392	0.013310	obs=-0.00275 (20.7% of MDE)
H4 control-DiD	Summer	Lights	SE=0.038911	0.117919	obs=+0.01798 (15.3% of MDE)

Sabse zaroori takeaway ye hai: H4 ke sabhi chaar combinations mein, observed coefficient apne MDE ka sirf 15% se 35% ke beech hai — matlab, jo bhi asli effect hai (agar hai bhi), wo detectable-threshold se bahut door, bahut chhota hai. Ye 'no effect detected' ko ek bahut zyada convincing statement banata hai — ye 'humara test weak tha' nahi, balki 'jo bhi effect ho sakta hai, wo genuinely bahut chhota hai, humare design ke apne detectable-threshold ke comfortably neeche.'

EXAMPLE — MDE Ko Concretely Samajhna — Lights Ke Liye

Full-year lights ka H1 MDE 21.2% hai apne baseline ka — matlab, agar VVP-I ka asli effect night-lights mein 21.2% se zyada ka increase hota, toh ye study use reliably (80% chance ke saath) pakad leti. Isse chhota koi bhi genuine effect miss ho sakta tha. Similarly, H4 full-year lights ka MDE 0.139825 hai, jabki observed coefficient sirf 0.03460 hai — matlab agar asli effect coefficient scale par 0.139825 se bada hota, wo pakda jaata; observed 0.03460 sirf uska 24.7% hai, jo batata hai ki agar koi genuine effect hai bhi, wo design ke detectable-range se kaafi neeche hai.

History — power_analysis.py Ka Apna, Chhota Lekin Zaroori Correction-Tail

EXAMPLE — Entry 39 — 721-Village List Ke Against Dobara Chalaya Gaya

Jab Entry 37 mein control list 732 se 721 tak district-verification ke saath aur sudhaari gayi, toh Development Log Entry 39 mein pata chala ki power_analysis.py ka output file (border_optics_power_analysis.json) ab bhi purani, 732-list numbers rakhta tha — is baar isliye nahi ki koi network-access chahiye thi (jo pretreatment placebo test ke case mein tha), balki sirf isliye ki script dobara chalayi hi nahi gayi thi, chahe usko chalane ke liye sirf already-available did_summary JSONs chahiye the. Isliye Entry 39 mein ise actually re-run kiya gaya (python3 src/analysis/power_analysis.py), sirf caveat lagane ke bajaye. Numbers materially badle: H1 ka Cohen's d 0.178 se 0.157 tak (ek script-fix ki wajah se, jo one-sided alternative testing sahi tarah use karne laga, jo already code ke apne comments mein

reflect ho raha tha, sirf output-file kabhi regenerate nahi hui thi is fix ke baad), aur H4 ka observed-as-fraction-of-MDE range '24 se 34%' se '15 se 35%' tak badla. Conclusion pe koi asar nahi pada — H4 ke sabhi observed effects abhi bhi comfortably apne detectable-threshold ke andar the — lekin exact numbers zaroor badle.

EXAMPLE — Entry 41 — Freeze Document Aur Dashboard Mein Purane Numbers Reh Gaye The

Entry 39 ne BO_Research_Paper.md ka §7.7 sahi update kar diya tha, lekin ANALYSIS_FREEZE.md ki apni power-analysis table (jo same numbers ko apni jagah alag se rakhti hai) aur pages/8_Methodology_Limitations.py ke dashboard ka power-analysis expander — dono miss ho gaye, ab bhi purane Cohen's $d=0.178$ aur '24-34%' range dikha rahe the. Entry 41 mein ek doosre, independent external review ne ise pakda, aur dono jagah current 721-village numbers ($d=0.157$, MDE 0.00802/0.09086/0.00531/0.07095, '15-35%') se sync kiya gaya. Ye bilkul PDF 16 mein dekhe gaye Holm-correction wale 'ek jagah fix kiya, doosri jagah bhool gaye' pattern jaisa hai — jab bhi ek number kai jagah quote hoti hai, ek jagah update karna kaafi nahi hota, poore repository mein systematic grep zaroori hota hai.

EXAMPLE — Entry 47 — Sidedness Ki Labeling Mein Ek Chhoti Prose-Galti

PDF 16 mein bhi Entry 47 ka zikar hua tha (Holm-vs-spatial correction ke context mein) — usi entry mein ek teesra, is file se directly related item bhi tha: paper ke §7.7 ne H1 aur H4, dono ke MDE ko ' $\alpha=0.05$, two-sided' keh diya tha. Lekin code khud (jo humne abhi mde_multiplier() mein poora padha) shuru se hi sahi tha — H1 one-sided (Wilcoxon ke apne alternative='greater' se match karte hue), H4 two-sided (statsmodels ke default two-sided p-values se match karte hue). Sirf paper ki prose ne dono ko galti se ek jaisa, 'two-sided' bata diya tha. Fix sirf wording tha — har test ki asli sidedness aur uski wajah state karne ke liye — code ya koi number badalne ki zaroorat nahi thi. Ye bilkul us pattern ka hissa hai jo is poore PDF mein baar-baar dikha: code khud accurate hota hai, lekin prose usse describe karte waqt kabhi-kabhi oversimplify kar deti hai, aur fix hamesha code ko paper ke against verify karke hi milta hai.

Teeno Parts Ko Jodkar Dekho

Is PDF ki teeno files, apne-apne tareeke se, is study ke null results ko zyada mazboot bunyaad par khadi karti hain. Part A ne dikhaya ki current null result (control-group DiD, dono windows) kisi ek district par depend nahi karta aur cluster-robust asymptotics ke bina bhi robust hai — lekin iske saath-saath ye bhi dikhaya ki yehi checks, apna kuch bhi badle bina, teen alag baar teen alag verdict de chuki hain, sirf isliye kyunki underlying data khud teen baar sudhaara gaya. Part B ne dikhaya ki 251-village core sample, size ke hisaab se toh unbiased hai, lekin district ke hisaab se

nahi — aur ye imbalance khaas taur par H3 ke apne border-proximity hypothesis ke liye relevant hai. Part C ne dikhaya ki H1 aur H4 ke null results sirf ' $p>0.05$ ' nahi hain — ye 'design apna detectable-threshold ka sirf 15-35% observe kar raha hai' jaisi ek zyada precise, zyada convincing statement hain.

Development Log ki Entry 39/41 ki 'ek jagah fix kiya, doosri jagah bhool gaye' kahani, aur Part A ki teen-baar-reversal wali kahani, dono ek hi underlying discipline dikhati hain jo is poore project mein baar-baar dikhi hai — data khud alag se, systematically verify karna, aur jab wo badle, har jagah jahan uska number quote hota hai, wahan tak pahunchna, sirf ek jagah patch karke aage badh jaana nahi.

RECAP — PDF 17 Mein Kya Seekha: (1) Leave-one-district-out aur randomization inference — do tareeke check karne ke ki koi result kisi ek cluster ya kisi asymptotic-assumption par zyada depend karta hai ya nahi. (2) `extended_robustness_checks.py` ki teen-baar-reversal history — checks khud kabhi galat nahi the, lekin unke neeche ka data teen baar badla (Sikkim missing→included, control-list contaminated→clean), aur har baar checks ne faithfully wahi verdict diya jo us waqt ke data ke hisaab se sahi tha. (3) `log1p` transformation — skewed data ko outlier-resistant banane ka ek standard tareeka. (4) Geocoding-selection-bias check — sample size ke hisaab se unbiased, lekin district ke hisaab se nahi, aur ye H3 ke border-proximity hypothesis ke liye khaas taur par relevant hai. (5) Minimum Detectable Effect (MDE) — ' $p>0.05$ ' se aage jaakar, 'itna chhota effect bhi detect ho jaata' jaisi precise statement dena, sidedness ko asli test se match karte hue (H1 one-sided, H4 two-sided). (6) Cohen's d ek design-property hai, outcome ki apni scale par depend nahi karta.

Aage, PDF 18 mein `building_footprint_validation.py`, `dw_sar_ndbi_village_level_check.py`, aur `triangulation_analysis.py` dekhenge — ye teeno files poochti hain ki NDBI, SAR, aur Dynamic World, teen bilkul alag satellite-based proxies, ek-doesre se kitna agree karte hain, aur jahan disagree karte hain, wahan kyun.